

MTE 544 – Assignment 2 Report

Prepared for Prof. Yue Hu

by Will McIntyre (21009235)

Question 1

Question 1 a)

Begin by looking at the quadratic equation representing the ellipse:

$$Ax^2 + 2Bxy + Cy^2 - 2Dx - 2Ey + 1 = 0$$

There are 5 unique coefficients, or 5 degrees of freedom, meaning that we need 5 distinct points to generate 5 independent linear equations to solve for them. For each point, we substitute the x and y values, from $p_i = (x_i, y_i)$ into the equation and move the constant to the right-hand side:

$$A(x_i)^2 + 2B(x_i y_i) + C(y_i^2) + D(-2x_i) + E(-2y_i) = -1$$

If we do this for all 5 points and construct a 5x5 matrix representation of the linear equations, we can solve for the coefficients, defining $M \in \mathbb{R}^{5x5}$ representing the linear equations, $v \in \mathbb{R}^{5x1}$ as the variable vector and $k \in \mathbb{R}^{5x1}$ as the constants:

$$Mv = k$$

$$\begin{bmatrix} x_1^2 & 2x_1y_1 & y_1^2 & -2x_1 & -2y_1 \\ x_2^2 & 2x_2y_2 & y_2^2 & -2x_2 & -2y_2 \\ x_3^2 & 2x_3y_3 & y_3^2 & -2x_3 & -2y_3 \\ x_4^2 & 2x_4y_4 & y_4^2 & -2x_4 & -2y_4 \\ x_5^2 & 2x_5y_5 & y_5^2 & -2x_5 & -2y_5 \end{bmatrix} \begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix}$$

As long as all the 5 points p_i are distinct and non-collinear we can compute the unique parameters by taking the inverse of the matrix.

$$\begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \begin{bmatrix} x_1^2 & 2x_1y_1 & y_1^2 & -2x_1 & -2y_1 \\ x_2^2 & 2x_2y_2 & y_2^2 & -2x_2 & -2y_2 \\ x_3^2 & 2x_3y_3 & y_3^2 & -2x_3 & -2y_3 \\ x_4^2 & 2x_4y_4 & y_4^2 & -2x_4 & -2y_4 \\ x_5^2 & 2x_5y_5 & y_5^2 & -2x_5 & -2y_5 \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix}$$

Question 1 b)

To fit an ellipse to the points given in the report, we sub them into the matrix and solve for coefficients $A. B, C, D$, and E:

$$(2.92, -6.01), (3.40, -7.20), (4.99, -7.84), (5.48, -7.04), (4.20, -5.91)$$

$$\begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \begin{bmatrix} 2.92^2 & 2(2.92)(-6.01) & (-6.01)^2 & -2(2.92) & -2(-6.01) \\ 3.40^2 & 2(3.40)(-7.20) & (-7.20)^2 & -2(3.40) & -2(-7.20) \\ 4.99^2 & 2(4.99)(-7.84) & (-7.84)^2 & -2(4.99) & -2(-7.84) \\ 5.48^2 & 2(5.48)(-7.04) & (-7.04)^2 & -2(5.48) & -2(-7.04) \\ 4.20^2 & 2(4.20)(-5.91) & (-5.91)^2 & -2(4.20) & -2(-5.91) \end{bmatrix}^{-1} \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ -1 \end{bmatrix}$$

We can then use any equation solver, in this case MATLAB, to solve for the coefficients:

$$\begin{bmatrix} A \\ B \\ C \\ D \\ E \end{bmatrix} = \begin{bmatrix} 0.0181 \\ 0.0121 \\ 0.0302 \\ -0.0062 \\ -0.1544 \end{bmatrix}$$

To solve for **Q** and **b**, first determine α:

$$\alpha = \frac{1}{\begin{bmatrix} D & E \end{bmatrix} \begin{bmatrix} A & B \\ B & C \end{bmatrix}^{-1} \begin{bmatrix} D \\ E \end{bmatrix} - 1}$$

$$\alpha = \frac{1}{\begin{bmatrix} -0.0062 & -0.1544 \end{bmatrix} \begin{bmatrix} 0.0181 & 0.0121 \\ 0.0121 & 0.0302 \end{bmatrix}^{-1} \begin{bmatrix} -0.0062 \\ -0.1544 \end{bmatrix} - 1}$$

$$\alpha = 41.6781$$

Now, solve for **Q:**

$$\boldsymbol{Q} = \alpha \begin{bmatrix} A & B \\ B & C \end{bmatrix}$$
$$\boldsymbol{Q} = 41.6781 \begin{bmatrix} 0.0181 & 0.0121 \\ 0.0121 & 0.0302 \end{bmatrix}$$
$$\boldsymbol{Q} = \begin{bmatrix} 0.7528 & 0.5030 \\ 0.5030 & 1.2567 \end{bmatrix}$$

Then, solve for **b:**

$$\boldsymbol{b} = \boldsymbol{Q}^{-1} \alpha \begin{bmatrix} D \\ E \end{bmatrix}$$
$$\boldsymbol{b} = \begin{bmatrix} 0.7528 & 0.5030 \\ 0.5030 & 1.2567 \end{bmatrix}^{-1} (41.6781) \begin{bmatrix} -0.0062 \\ -0.1544 \end{bmatrix}$$
$$\boldsymbol{b} = \begin{bmatrix} 4.1987 \\ -6.8015 \end{bmatrix}$$

Question 2

Question 2.a

Using the process described in Question 1, *fit_ellipse_subset* was written in the Python code to compute $\boldsymbol{Q}$ and $\boldsymbol{b}$ from a given set of 5 points. It takes in 5 points, separates the x and y components, and then constructs the M matrix and constant vector k. Coefficients are solved for using numpy, which are then used to find alpha, Q and b using the formulas described in Question 1.

Question 2.b

The implementation to determine the best $\boldsymbol{Q}, \boldsymbol{b}$ and their corresponding inliers picks a random selection of 5 points from the dataset to fit to an ellipse. To ensure that Q represents an ellipse, it must be positive definite. It was noted that sometimes the fitting produces centers $\boldsymbol{b}$ that are extremely far away from the data when the 5 random points sit in an almost straight line. When this occurs, the ellipse is huge and all the data is included in the inlier set due to its size. To prevent this, any $\boldsymbol{b}$ fit that is far away from the dataset (outside the range -20 to 20 in x/y) is ignored. All points within the desired threshold from the fitted ellipse are deemed inliers and tracked. This process is repeated for the number of desired iterations, and the $\boldsymbol{Q}$ and $\boldsymbol{b}$ that produce the most inliers are output and plotted.

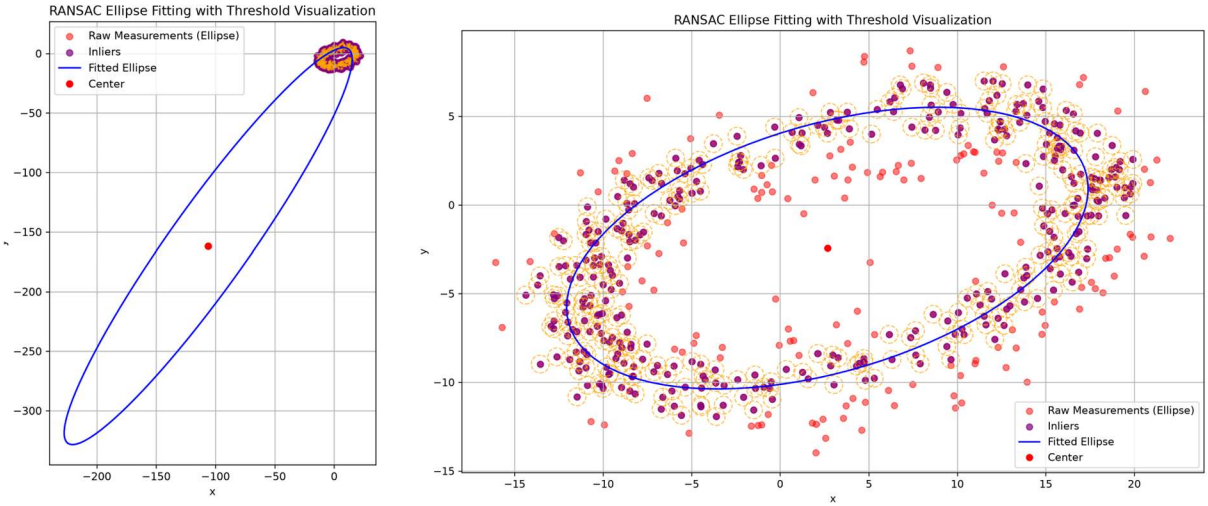


Figure 1: RANSAC Before Center Filtering (Left) and After (Right)

Question 2.c

Reducing the number of points in the dataset makes the ellipse shape ambiguous, while a dense dataset clearly defines the ring.

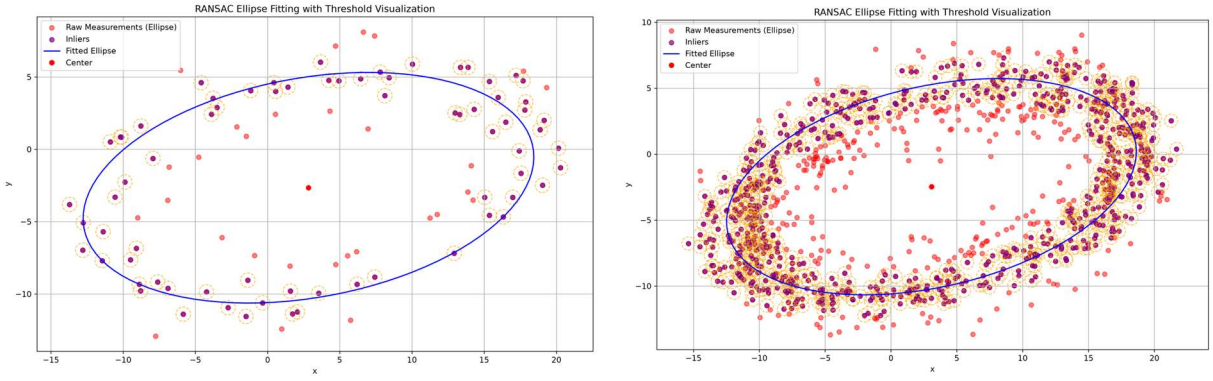


Figure 2: RANSAC With Dataset Size of 100 (Left) vs 1000 (Right)

Looking at the number of iterations, a low number can cause failure to find the ellipse and return stretched shapes. As the number of iterations increase, the algorithm consistently locks into the ellipse shape.

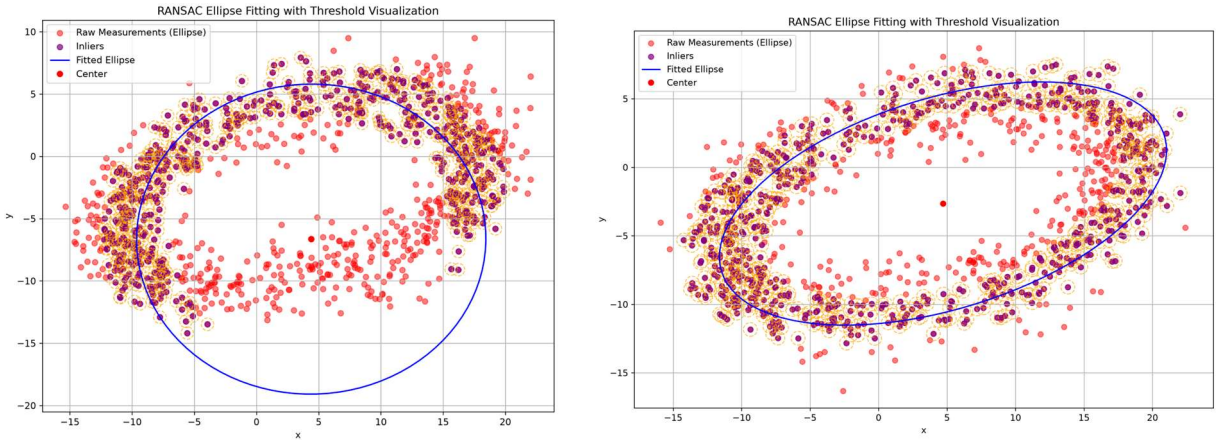


Figure 3: RANSAC With 5 Iterations (Left) vs 100 Iterations (Right)

Finally, a lower threshold demands tighter precision, which must be paired with a large number of iterations to get favourable results. On the other hand, a higher threshold accepts more sensor noise but risks sweeping up false positives and can generate inaccurate ellipses as well.

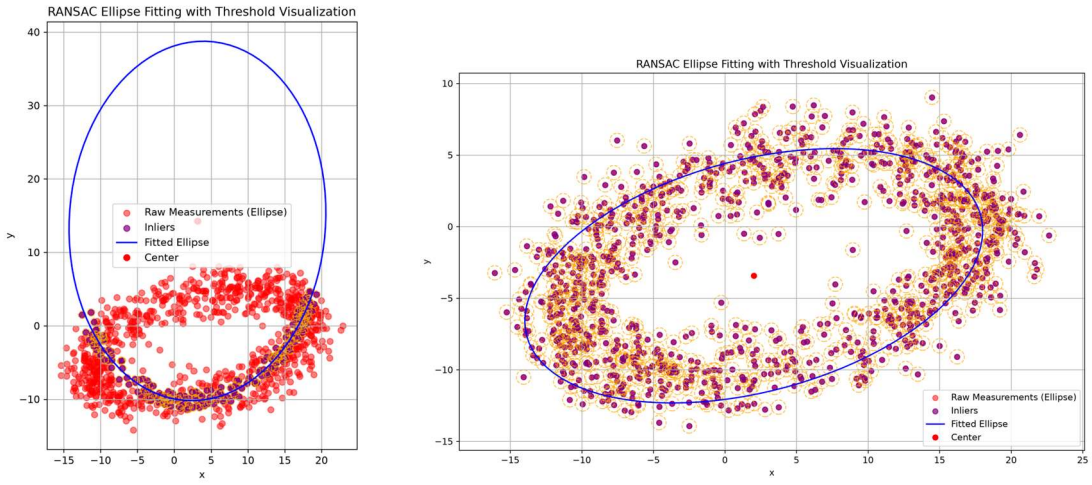


Figure 4: RANSAC With Tight Tolerance of 0.05 With 1000 Iterations (Left) vs Tolerance of 0.8 and 1000 Iterations.

Question 3

Question 3.a)

The total probability is the sequential Markov state transitions starting from $A_0 = 1$. We multiply each of the conditional probabilities together to find the likelihood of the exact path.

$$P(A_1 = 2|A_0 = 1) = 0.4$$
$$P(A_2 = 3|A_1 = 2) = 0.4$$
$$P(A_3 = 3|A_2 = 3) = 0.7$$
$$P(A_4 = 1|A_3 = 3) = 0.2$$

So, the total probability:

$$P = 0.4 \times 0.4 \times 0.7 \times 0.2 = 0.0224$$

The total probability of Moo-Deng following the sequence (2,3,3,1) is 2.24%.

Question 3.b)

The transition matrix is constructed such that entry $A_{i,j}$ represents the probability of moving to area i given the current location is area j. Each column sums to 1, satisfying the conservation of probability across the equation $x_{k+1} = Ax_k$.

For example, $A_{1,1}$ represents $P(x_{k+1} = 1|x_k = 1) = 0.6$.

$$A = \begin{bmatrix} 0.6 & 0.2 & 0.2 \\ 0.4 & 0.4 & 0.1 \\ 0 & 0.4 & 0.7 \end{bmatrix}$$

Question 3.c)

To find the stationary distribution $\boldsymbol{x}^*$, we know the following must hold:

$$x^* = Ax^* \quad (1)$$
$$x_1^* + x_2^* + x_3^* = 1 \quad (2)$$

Simplifying (1):

$$(A - I)x^* = 0$$
$$\begin{bmatrix} 0.6 - 1 & 0.2 & 0.2 \\ 0.4 & 0.4 - 1 & 0.1 \\ 0 & 0.4 & 0.7 - 1 \end{bmatrix} \begin{bmatrix} x_1^* \\ x_2^* \\ x_3^* \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Extracting the system of linear equations:

$$-0.4x_1^* + 0.2x_2^* + 0.2x_3^* = 0 \quad (3)$$

$$0.4x_2^* - 0.3x_3^* = 0 \quad (4)$$

Solving for x_2^* using (4) and subbing it into (3):

$$x_2^* = \frac{3}{4}x_3^*$$
$$-0.4x_1^* + 0.2\left(\frac{3}{4}x_3^*\right) + 0.2x_3^* = 0 \Rightarrow x_1^* = \frac{7}{8}x_3^*$$

Then, applying to probability constraint (2):

$$\frac{7}{8}x_3^* + \frac{3}{4}x_3^* + x_3^* = 1$$
$$x_3^* = \frac{8}{21}$$

We can then find x_1^* and x_2^*:

$$x_1^* = \frac{7}{8}x_3^* = \frac{7}{8}\left(\frac{8}{21}\right) = \frac{7}{21}$$

$$x_2^* = \frac{3}{4}x_3^* = \frac{3}{4}\left(\frac{8}{21}\right) = \frac{2}{7}$$

The final stationary probabilities of Moo-Deng's location follow:

$$\boldsymbol{x}^* = \begin{bmatrix} x_1^* \\ x_2^* \\ x_3^* \end{bmatrix} = \begin{bmatrix} \frac{7}{21} \\ \frac{2}{7} \\ \frac{8}{21} \end{bmatrix} \approx \begin{bmatrix} 0.333 \\ 0.286 \\ 0.381 \end{bmatrix}$$

Question 3.d)

The matrix C is constructed such that $C_{i,j}$ represents the probability of sensor reporting area i when Moo-Deng is in area j. The columns sum to 1 to satisfy the conservation of probability.

For example, $C_{1,1}$ represents $P(y_k = [1 \quad 0 \quad 0]^T | x_k = 1) = 0.8$.

$$C = \begin{bmatrix} 0.8 & 0.2 & 0.05 \\ 0.1 & 0.7 & 0.1 \\ 0.1 & 0.1 & 0.85 \end{bmatrix}$$

Question 3.e)

Deriving a state estimator based on a Bayesian filter that estimates the states and probability of Moo-Deng's based on prior belief $\boldsymbol{x_{k-1}}$ and the current sensor measurement

$\boldsymbol{y_k}$, begins with the prediction step. Using the previous belief $\boldsymbol{x_{k-1}}$ and the transition matrix A, the predicted state distribution follows:

$$\bar{x}_k = Ax_{k-1}$$

Then, the update step is based on the sensor measurement $\boldsymbol{y_k}$ and the predicted state. The likelihood of this specific observation across all three states can be extracted by multiplying the transpose of the measurement matrix by the observation. We take the transpose to isolate the row in the measurement matrix given by a sensor reading y_k. The update step refines the predicted state using the measurements from sensors and element wise multiplication of the prior prediction:

$$x_k = \eta p(y_k|x_k)\bar{x}_k$$
$$x_k = \eta(C^T y_k) \circ \bar{x}_k$$

To solve for η, we can sub the elements of the unnormalized state vector and take the inverse, ensuring the final probability distribution sums exactly to 1:

$$\sum x_k = 1$$
$$\sum(\eta(C^T y_k) \circ \bar{x}_k) = \eta\sum\big((C^T y_k) \circ \bar{x}_k\big) = 1$$
$$\eta = \frac{1}{\sum\big((C^T y_k) \circ \bar{x}_k\big)}$$

So, the final state estimator equation follows:

$$x_k = \frac{(C^T y_k) \circ \bar{x}_k}{\sum\big((C^T y_k) \circ \bar{x}_k\big)} = \frac{(C^T y_k) \circ Ax_{k-1}}{\sum\big((C^T y_k) \circ Ax_{k-1}\big)}$$

Question 4

Question 4.a)

The implementation takes the transition probabilities given the current state from matrix A. Then, it uses *np.random.choice* to sample the next state. To verify statistically, run the simulation continuously for thousands of iterations; the overall frequency of visited states will converge to the computed stationary distribution x^* from Question 3.

Question 4.b)

The implementation takes the sensor probabilities given the current state from matrix C. Then, it also uses np.random.choice to sample the reading and then converts it to the one-hot encoding format.

Question 4.c)

The *sim_moodeng* function implements the Bayesian filter inside an iterative loop to generate the required sequence of state estimators $\hat{A}_k$ and belief vectors x_k. It follows the steps outlined in Question 3 by determining the prior prediction, the measurement likelihood and then combines them into a belief vector. Then, the most probable state is selected as the estimated state at that specific instant.

An example of the output follows:

Table 1: Example Bayesian Filter Output for 20 Iterations

Iteration (k)	Estimated State ($\hat{A}_k$)	p(Feeding) [Zone 1]	p(Resting) [Zone 2]	p(Playground) [Zone 3]
1	2	0.1190	0.7500	0.1310
2	1	0.6835	0.2489	0.0676
3	2	0.1444	0.8108	0.0448
4	3	0.0703	0.1054	0.8243
5	1	0.7479	0.1252	0.1269
6	3	0.2445	0.1773	0.5782
7	2	0.1262	0.6721	0.2016
8	3	0.0615	0.0833	0.8552
9	1	0.7488	0.1196	0.1317
10	1	0.8348	0.1506	0.0146
11	1	0.8379	0.1552	0.0069
12	1	0.8377	0.1557	0.0065
13	1	0.8377	0.1558	0.0065
14	1	0.8377	0.1558	0.0065
15	1	0.8377	0.1558	0.0065
16	2	0.1579	0.8223	0.0197
17	3	0.0737	0.1104	0.8159

18	2	0.1188	0.5626	0.3186
19	1	0.7039	0.2164	0.0796
20	3	0.2329	0.1819	0.5851